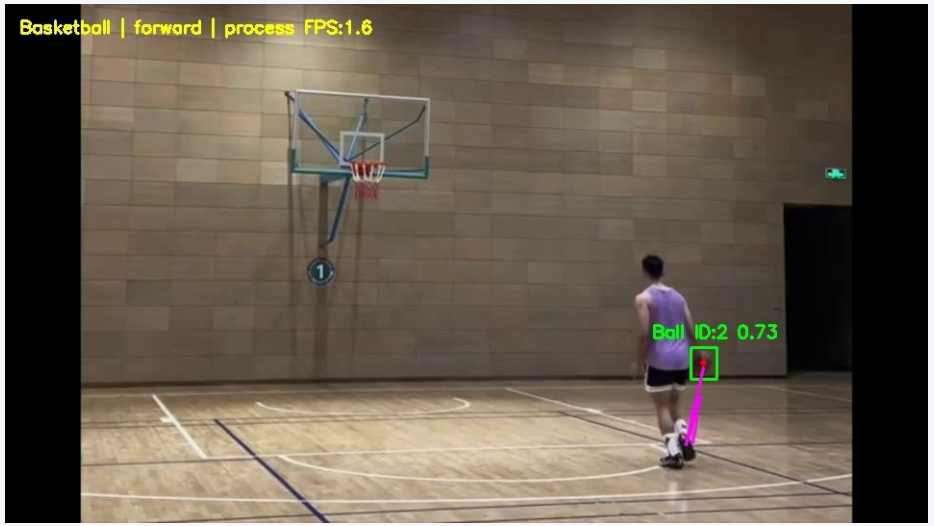
基于 YOLO 与 ByteTrack 的篮球检测与运动轨迹跟踪系统

从目标检测，到身份关联，再到短时遮挡预测



Python 3.11 / 3.12 · OpenCV · Ultralytics

CPU 优先，支持配置 CUDA 设备

01 要解决什么问题?

当前实现范围

输入：本地篮球视频

输出：球框、中心点、目标 ID 、历史轨迹

附加：处理视频、逐帧记录、短时预测

主要困难

篮球小、运动快、容易模糊

身体遮挡与人头外观相似

转播镜头平移、缩放和切换

当前重点：每帧选择一个主篮球

02 系统整体流程

视频读取

video.py

场景与检测

scene / basketball / closeup

候选筛选

pose / head_memory

身份关联

内置 ByteTrack

轨迹与预测

motion / display_trail / occlusion

真实检测 → tracks.csv | 预测 → predictions.csv | 可视化 → annotated.mp4

03 YOLOv8n：先找到“球在哪里”

原理与做法

单阶段检测：一次前向计算输出候选框

骨干网络提取边缘、纹理与语义特征

多尺度特征融合，兼顾不同大小目标

检测头输出位置与类别分数， NMS 去重

优势与边界

n 版本模型小，适合 CPU 起步

利用预训练权重，减少训练成本

篮球很小时，细节仍可能不足

置信度是模型评分，不等于准确率

04 当前到底用了什么权重?

原理与做法

basketball_base.pt： 公开篮球模型

basketball_closeup.pt： 近景微调权重

basketball.pt： 转播场景适配权重

yolov8n-pose.pt： 人体关键点辅助

优势与边界

当前是篮球专用权重，球类别为 ball

与 COCO 通用 sports ball 版本不同

近景微调： 24 帧训练、 6 帧开发验证

相邻同视频验证，不能代表跨场景准确率

05 ByteTrack：把连续帧中的球关联起来

原理与做法

卡尔曼滤波先预测目标下一位置

第一轮：用高分检测匹配已有轨迹

第二轮：用低分检测补充匹配

IoU 代价与线性分配维护目标 ID

优势与边界

可利用遮挡后得分下降的真实检测

计算开销小，无额外 ReID 网络

直接使用 Ultralytics 内置实现

误检持续出现，也可能被稳定跟踪

06 小篮球快速运动，为什么容易断 ID？

原理与做法

普通 IoU：前后球框不重叠就难匹配

内部扩大关联框，容忍更大的位移

真实检测框仍按原尺寸绘制和导出

状态：位置、宽高比、高度及其速度

优势与边界

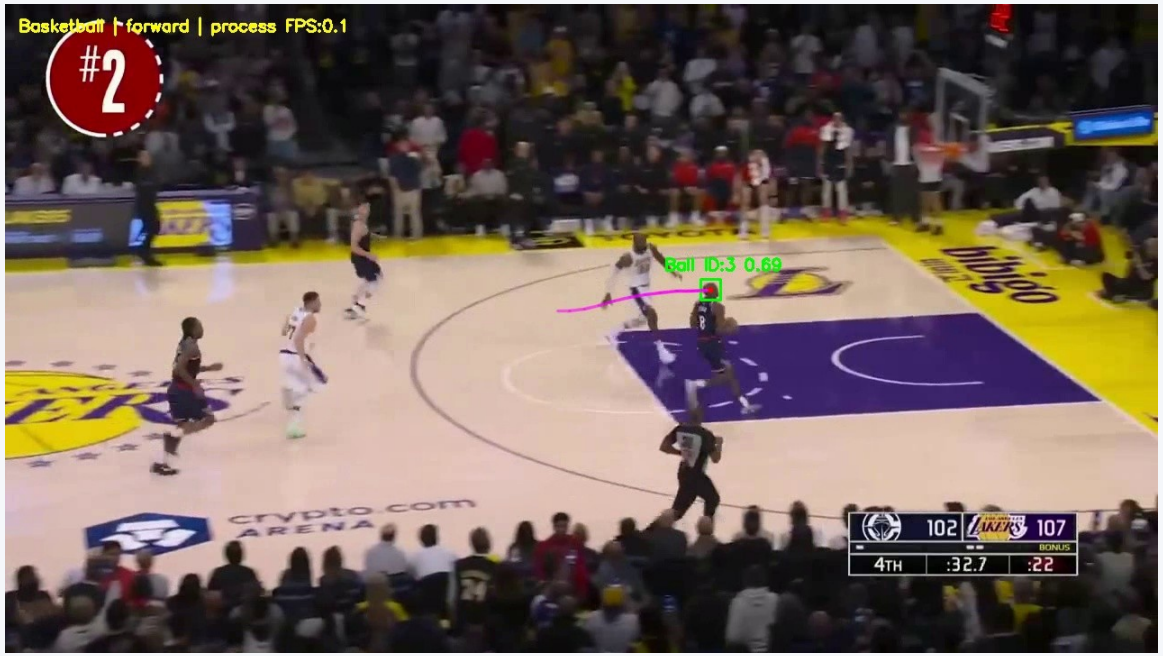
优点：减少小目标高速移动时断轨

代价：扩大窗口增加错误关联风险

track_buffer = 45 帧（30 FPS 约 1.5 秒）

match_thresh = 0.99 是代价阈值

07 人头误检：引入人体姿态与语义约束



旧配置：人头可能成为候选球



修正：场景适配 + 头部区域排除

取舍：排除过强会误伤贴近脸部的真球；约束过弱则容易误检。

08 漏检恢复与自动场景选择

原理与做法

全帧搜索 → 局部 ROI → 人体周围补检

多尺度输入保留小球细节

结合大小、外观、运动一致性选候选

所有观测框仍来自 YOLO

优势与边界

启动时采样人体数量与相对高度

近景：优先单向处理

转播：可做离线双向候选选择

只在开头判断，不能完全应对后续切镜

09 相机运动补偿：轨迹跟着场景移动

原理与做法

背景角点： Shi–Tomasi

连续帧跟踪： LK 光流与双向一致性

鲁棒估计： RANSAC 部分仿射变换

将历史点映射到当前帧再绘制

优势与边界

$p_{当前} = A \times p_{上一帧} + b$

抵消平移、旋转和近似统一缩放

减少运镜造成的屏幕“乱线”

不能恢复三维轨迹，也不是球场坐标

10 轨迹显示：短、清楚、与观测区分

原理与做法

仅显示最近 0.7 秒轨迹

旧线段淡出，显示中心点做 EMA 平滑

平滑权重 0.8 ，优先保留当前位置

漏检、跳变、换 ID 时断开线段

优势与边界

绿色框 / 红色点：真实检测

紫色实线：历史观测轨迹

橙色虚线 / 空心点：预测位置

原始坐标记录不被显示平滑替换

11 遮挡预测：只预测短时间，承认不确定性

原理与做法

取最近最多 6 个真实观测点

至少 3 个连续点：拟合线性运动

点数充足：二次最小二乘拟合

默认最多向前预测 0.3 秒

优势与边界

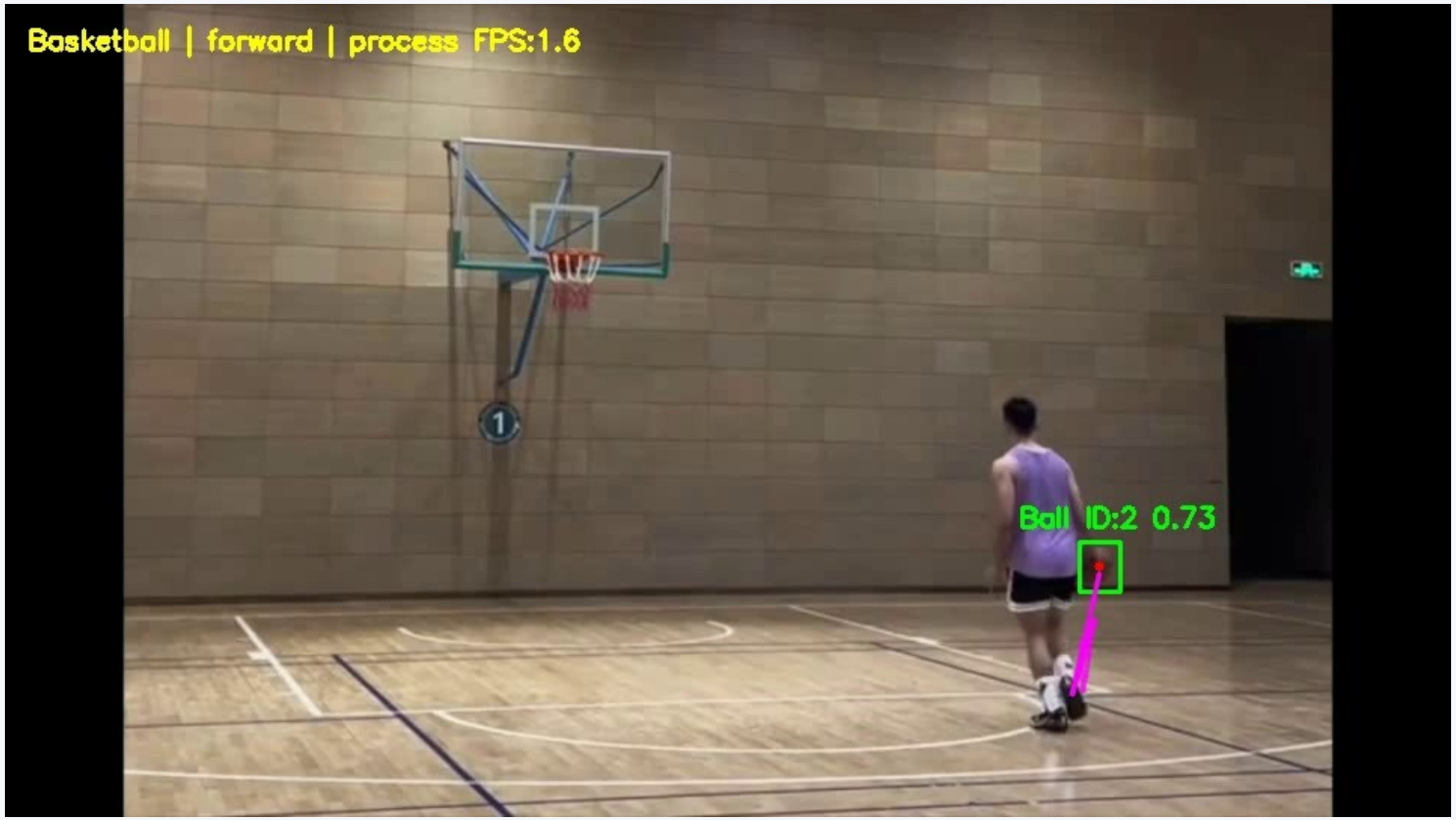
碰头区、出画、拟合误差大时停止

切镜或再出现位置异常时不强连

预测单独记录，不反馈到 ByteTrack

像素拟合，不是三维物理抛物线

12 当前效果与可重复输出



main.py 修改视频路径

选择场景与处理模式

运行后检查视频与 CSV

记录模型、参数、环境、耗时

13 开发实验：有改善，但还不是独立测试成绩

项目	优化前	优化后
近景 300 帧：真实观测帧	177	277
确认跟踪帧	156	268
输出过的不同 ID 数	4	3
NBA 可见球抽检	6 / 8	7 / 8
NBA 人头 / 篮筐负样本误报	3 / 9	0 / 9

277 / 300 = 92.3% 是观测输出覆盖率，不是准确率、召回率或 mAP 。

14 还需要怎样评估，才能说明“跟得准”？

原理与做法

独立视频逐帧标注球框与身份

检测： Precision 、 Recall 、中心点误差

跟踪： IDF1 、身份切换与断轨

遮挡前 / 中 / 后分别统计

优势与边界

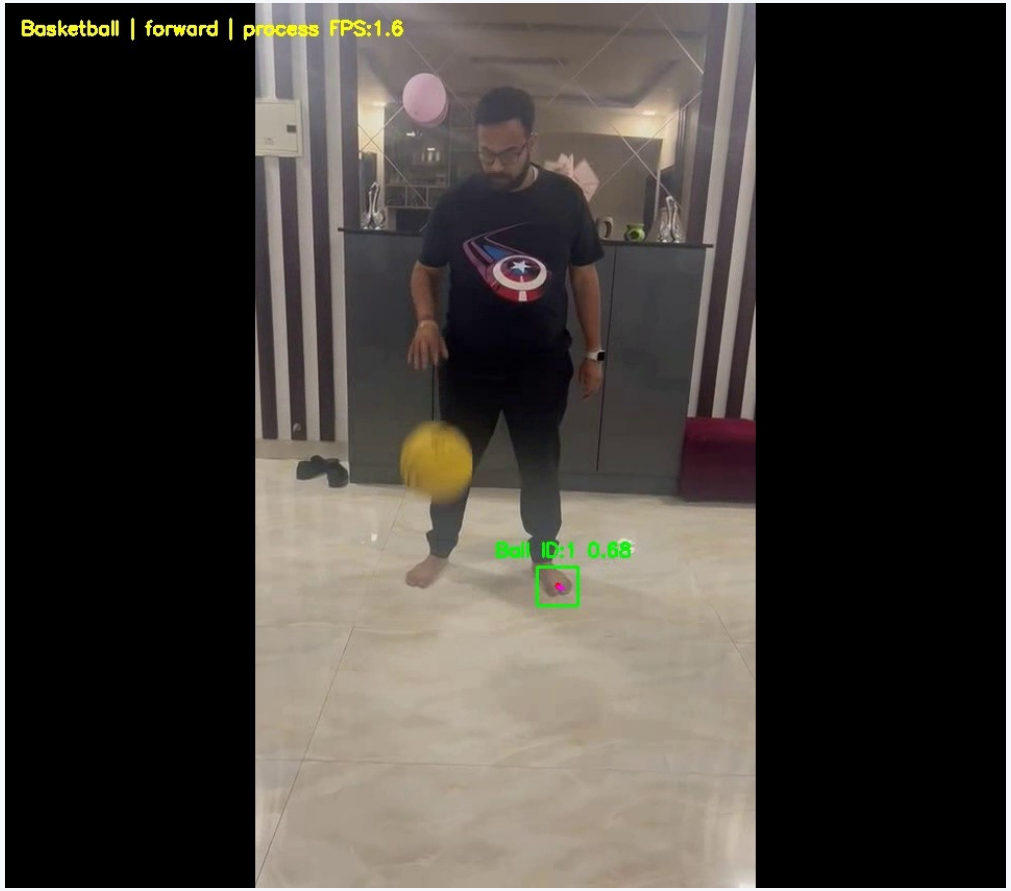
固定模型、阈值、分辨率和设备

记录端到端 FPS 与纯推理耗时

Python 3.11 / 3.12 各 43 项测试通过

单元测试通过 ≠ 实际视觉效果合格

15 当前反例：稳定 ID，也可能一直跟错



新场景 75 / 75 帧有输出

只出现 1 个目标 ID

但真实篮球在其他位置

结论：必须对照真值检查！

16 下一步改进：先提升检测，再改进预测

原理与做法

优先级 1： 独立数据与困难负样本

加入人头、手臂、鞋、篮筐等反例

补充持球、模糊、远景与不同颜色球

按视频划分训练 / 验证 / 测试集

优势与边界

优先级 2： 更稳的时序约束

引入持球 / 飞行 / 反弹状态判断

量化预测不确定性与再出现匹配

优先级 3： 测速、 ROI 调度、模型加速

17 项目总结与答辩要点

算法分工

YOLO：提供球的真实位置观测

ByteTrack：关联观测并维护 ID

姿态与语义：减少相似物体误检

光流补偿：改善运镜下的轨迹显示

目前能得出的结论

已实现视频处理、导出与短时预测

开发样例表现有所提升

仍有漏检、误检和跨视频泛化问题

连续的线，不代表正确的球轨迹

附录 A 代码定位与关键参数

原理与做法

main.py： 路径、设备、场景开关

video.py： 读取、调度、结果导出

closeup.py / basketball.py： 检测筛选

head_memory.py： 头部区域记忆

优势与边界

motion.py： 相机运动估计

display_trail.py： 0.7 秒历史轨迹

occlusion.py： 0.3 秒短时预测

new_tracker()： 内置 ByteTrack 参数

附录 B 参考资料与复现依据

Ultralytics YOLOv8 模型说明

https://docs.ultralytics.com/models/yolov8/

Ultralytics 跟踪模式与内置跟踪器

https://docs.ultralytics.com/modes/track/

Zhang et al. · ByteTrack · ECCV 2022

https://arxiv.org/abs/2110.06864

OpenCV 光流与运动估计文档

https://docs.opencv.org/4.x/dc/d6b/group__video__track.html

本项目源码、权重来源与开发测试记录

https://github.com/q111v/ball-tracking-project/tree/pycharm-prediction